Testing and Test Design Framework for Sift

Test strategy, results, coverage and defect register, aligned to the US DoD CDAO, Test and Evaluation of AI Models (April 2024)

PROJECT	DELIVERABLE	TEAM NO	TEAM MEMBER	FRAMEWORK
Sift	**Deliverable 2: Testing**	**Team 24 (single member)**	**Kundhave S, CB.SC.U4CSE23037**	**DoD CDAO T&E of AI Models**

1. Executive Summary

The suite has **220 test items: 214 passed, 6 xfailed (known defects, strict) and 0 failed**, with 73.4% line+branch coverage of src/sift and app/backend (836 of 1137 statements; e2e excluded). It ran on x86_64, device cpu, on 2026-10-16. Tests are organised along the six sections of the US DoD CDAO, Test and Evaluation of AI Models (April 2024): Performance, Testing Methods, Data, AI Models, Context and Documentation (section 6 gives what was and was not done).

- **Findings:** six defects were found by the tests and are kept as `xfail(strict=True)` cases, so the suite goes red if one is silently fixed. The most serious (D3, High) is an adversarial one: echoing the intent text as a comment inside a misaligned hunk makes the model call it aligned in most attempts (section 3.6).
- **Metamorphic relations** held within tolerances that were fixed before the first run (section 3.5); determinism and batch independence are exact to float precision.
- **Not claimed:** the coverage figure excludes the training and experiment scripts, four modules are at 0% in the unit suite (explain, distill, gan_baseline, make_artifact), and everything ran on one CPU machine.

2. Responsibility Matrix

Activity	Owner	Artefacts
Test planning, design techniques, test code, execution, coverage, defect register, this report	All work by Kundhave S (single-member team)	tests/, e2e/, results/test_summary.json, results/coverage.json, experiments/results/testing_measurements.json

3. Testing Taxonomy and Test Design

Test types follow the framework's Testing Methods section. Each type below lists its design technique and the real result.

3.1 Smoke / readiness and 3.2 White-box structural

Smoke (6/6 passed): package imports, dataset loading, a random-init 2-layer tiny cross-encoder forward pass (so structure tests need no download), tokenizer availability, the trained artifact scoring a diff, API health. White-box (51/51 passed, including the hunk parser and intent consolidation): tensor shapes, the three head outputs, focal and multi-task loss values, truncation at the maximum length, NDCG/P@K/MRR on hand-computed cases, temperature scaling and ECE. Statement and branch coverage guided the choice of cases (section 4).

3.3 Black-box functional

Techniques: **equivalence partitions** (git-style vs plain diff, single vs multi-file, add-only vs delete-only vs mixed hunks, one vs many intent turns, supported vs unsupported language), **boundary values** (zero hunks, one hunk, a thousand hunks, empty and maximal intent/hunk lengths) and **error guessing** (CRLF, binary files, paraphrased pleasantries, unicode). 31 passed and 2 xfailed (D1 and D2). API partitions are in two further suites (43/43 passed).

3.4 Data pipeline

17 passed, 2 xfailed. Checks: split sizes (12,600 / 2,700 / 2,700), **zero repositories, sessions and intent-hunk pairs shared across splits** (holdout by repository), train label flips exactly 569, val/test labels clean, drift type and tier only on misaligned rows, CSV and JSONL agree, deterministic loading. The two xfails (D-DATA-1, D-DATA-2) are real discrepancies with the documented counts (section 5).

3.5 Metamorphic testing (trained ModernBERT, test-split samples)

ID	Relation (input change -> expected output)	Measured	Tolerance	Result
MR1	Same input twice -> identical probabilities	max |dp| = 0.0 (n=120)	exact	pass
MR2	Pair scored alone vs inside a batch -> same probability	max |dp| = 1.4e-06 (n=40)	float noise	pass
MR3	Reorder hunks within a session -> each score unchanged	max |dp| = 2.1e-06 (25 sessions)	float noise	pass
MR4	Add an unrelated context comment -> verdict unchanged	flip rate 3.3%, mean |dp| 0.020	flip rate <= 10%	pass
MR5	Trailing whitespace on a context line -> verdict unchanged	flip rate 4.2%, mean |dp| 0.020	<= 10%	pass
MR6	Rename a local identifier in hunk and intent consistently -> unchanged	flip rate 5.2%, max |dp| 0.48	<= 10%	pass
MR7	Remove chit-chat turns from the intent -> unchanged	flip rate 1.7%, mean |dp| 0.017	<= 5%	pass
MR8	Directional: swap in another session's intent -> mean p(misaligned) rises	0.191 -> 0.410	rises	pass

All eight relations are in test_metamorphic.py (8/8 passed). Larger changes in MR6 (max |dp| 0.48) show that individual hunks can move a lot even when the flip rate is within tolerance.

3.6 Adversarial / red-team

Attack (goal: detected-misaligned hunk flips to aligned unless stated)	Success rate, latest run	Limit	Result
A1 Comment line copying the intent text added to the hunk (D3)	68% (17/25)	20%	xfail (defect D3)
A2 Context comment 'this change implements: <intent>' (D4)	52% (13/25)	20%	xfail (defect D4)
A3 Prompt-injection string ('IGNORE ALL PREVIOUS INSTRUCTIONS ... aligned') as added comment	4% (1/25)	20%	pass
A4 Irrelevant comment on an aligned hunk (goal: flips to misaligned, false alarm)	2% (1/50)	20%	pass
A5 50,000-line hunk	model survived, p = 0.322	no crash	pass
A6 Very long intent, empty inputs, unicode/control characters, binary-like hunk, 1000-hunk diff	no crash, finite probabilities	no crash	pass

Reading: the model fails only when the attacker places the intent's own words inside the hunk. It is robust to generic injection strings and to irrelevant comments, which implies it partly keys on lexical overlap between intent and hunk (the same overlap the hard tier is mined on). The attack table and the defect register in section 5 report the same latest measured run (testing_measurements.json). The denominator is only the misaligned test hunks that were detected first, so these rates have wide uncertainty (about +/- 20 points at n=25).

3.7 Back-to-back (differential) testing and UI end-to-end

Comparison (n = 300 test pairs)	Measured	Result
Student PyTorch vs student ONNX FP32 (equivalence of export)	max |dp| = 1.5e-06	pass
Student FP32 vs INT8 verdict agreement	95%; mean |dp| 0.035, max 0.41	pass (threshold in test)
Full model (teacher) vs student verdict agreement	86.7%; probability correlation 0.711	pass (threshold in test)
Teacher vs student within-session rank correlation (Spearman)	0.708	pass

INT8 changes about 5% of verdicts relative to FP32, so quantisation is not free. UI end-to-end (Playwright, e2e/test_ui_e2e.py): 37/37 passed in the run behind test_summary.json; the file has since grown (20 test functions counted by grep) and the new ones were not re-aggregated here.

3.8 Test design techniques

- **Factor selection:** input factors were diff shape (files, hunks, add/delete mix, length), intent shape (turns, chit-chat, retraction), language and content type (code, comment, whitespace, binary-like). Selected factors were varied one at a time for metamorphic and red-team cases, and combined only in partitions of the API suite; full pairwise generation (framework term) was not used.
- **Holdout by repository:** evaluation uses the repository-disjoint test split (httpx, attrs, axios); the data pipeline tests assert that no repository, session or pair crosses splits.
- **Edge cases and low-density regions:** empty and 1,000-hunk diffs, 50,000-line hunks, truncation beyond the model length, and the hard tier of constructed misaligned hunks.
- **Thresholds before results:** metamorphic tolerances and the 20% attack limit were fixed before the first run and not tuned afterwards.

4. Results and Coverage

Suite (category)	Files	Passed	Xfailed	Failed	Tests
Smoke / readiness	test_smoke.py	6	0	0	6
White-box structural	test_diffparse_consolidate.py, test_whitebox.py	51	0	0	51
Black-box functional	test_blackbox.py	31	2	0	33
Data pipeline	test_data_pipeline.py	17	2	0	19
Metamorphic	test_metamorphic.py	8	0	0	8
Adversarial / red-team	test_redteam.py	17	2	0	19
Back-to-back	test_backtoback.py	4	0	0	4
API (partitions)	test_api.py, test_api_partitions.py	43	0	0	43
UI end-to-end	test_ui_e2e.py	37	0	0	37
Total		**214**	**6**	**0**	**220**

Suite contents

File	What it tests
test_smoke.py	imports, dataset load, tiny-model forward, tokenizer, artifact scores a diff, API health
test_whitebox.py	tensor shapes, three head outputs, losses, tokenisation limits, ranking metrics, calibration
test_diffparse_consolidate.py	unified-diff hunk parser and intent consolidation rules
test_blackbox.py	equivalence partitions, boundary values, error guessing on diffs and intents
test_data_pipeline.py	split sizes, zero shared repositories/sessions/pairs, label noise counts, determinism
test_metamorphic.py	eight output relations (MR1-MR8) on the trained model
test_redteam.py	attack success rates, long/empty/unicode/binary-like inputs
test_backtoback.py	full model vs student vs ONNX FP32 vs INT8
test_api.py	FastAPI endpoints (health, analyse, explain, results)
test_api_partitions.py	request partitions and malformed-request handling of the API
e2e/test_ui_e2e.py	Playwright: UI flows, persistence, results pages

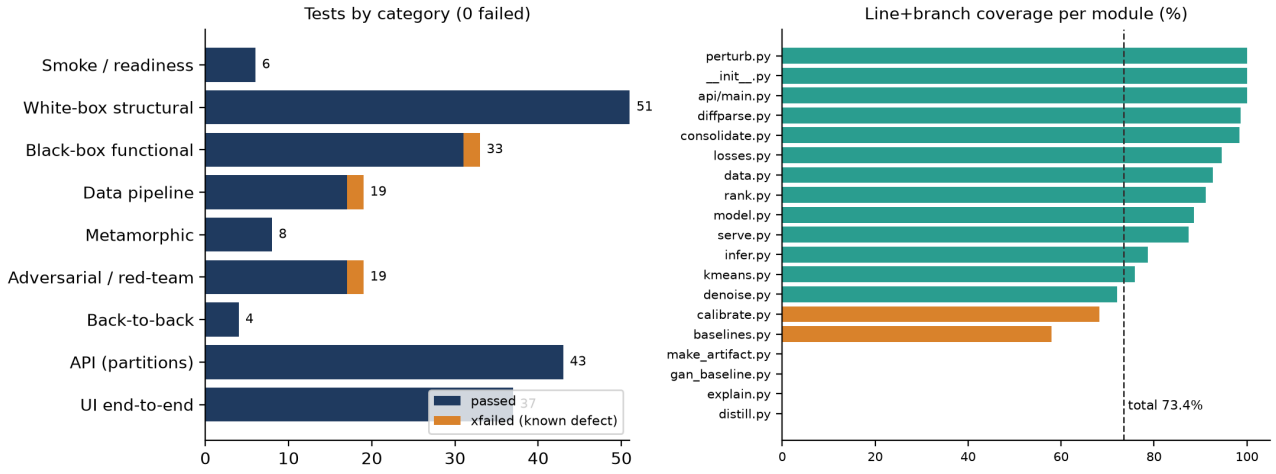


Figure 1: tests by category (left) and per-module coverage (right), from test_summary.json and coverage.json.

Module	Statements	Covered	Coverage
src/sift/distill.py	38	0	0.0%
src/sift/explain.py	74	0	0.0%
src/sift/gan_baseline.py	64	0	0.0%
src/sift/make_artifact.py	20	0	0.0%
src/sift/baselines.py	49	29	57.9%
src/sift/calibrate.py	22	15	68.2%
src/sift/denoise.py	25	18	72.0%
src/sift/kmeans.py	46	35	75.9%
src/sift/infer.py	88	71	78.7%
src/sift/serve.py	318	289	87.5%
src/sift/model.py	31	29	88.6%

Module	Statements	Covered	Coverage
src/sift/rank.py	80	72	91.2%
src/sift/data.py	35	33	92.7%
src/sift/losses.py	31	30	94.6%
src/sift/consolidate.py	44	44	98.4%
src/sift/diffparse.py	95	94	98.6%
app/backend/main.py	53	53	100.0%
src/sift/__init__.py	1	1	100.0%
src/sift/perturb.py	23	23	100.0%
Total (src/sift + app/backend (unit/API tests; e2e not measured))	**1137**	**836**	**73.4%**

Coverage excludes experiments/ and training scripts. Modules at 0% (explain, distill, gan_baseline, make_artifact) are exercised by the experiment scripts and the UI, not by unit tests. Per-module percentages in coverage.json include branch coverage; totals are the pytest-cov figures.

5. Defect Register

Only defects found by the tests are listed. Each is a strict xfail: the test asserts the correct behaviour and is expected to fail until the defect is fixed. All six are open.

ID	Description	Severity	How found	Mitigation / status
D3	comment line copying the intent text added to a misaligned hunk flips 68% (17/25) of detected hunks to 'aligned'; threshold 20%	High	Red-team: comment line copying the intent text added to a misaligned hunk	Open. Model partly keys on lexical overlap with the intent. Mitigation: strip comment-only added lines before scoring (not implemented) or adversarial training.
D4	context comment 'this change implements: <intent>' flips 52% (13/25) of detected hunks to 'aligned'; threshold 20%	Medium	Red-team: context comment claiming 'this change implements: <intent>'	Open. Same root cause as D3. Mitigation: ignore comments claiming intent; flag hunks whose only intent-overlap is in comments.
D1	plain `diff -u` multi-file output (no `diff --git` lines) merges the second file's header into the previous hunk	Medium	Black-box test on plain `diff -u` multi-file input (no `diff --git` lines)	Open: only git-style diffs split files correctly. Mitigation: UI/docs state git unified diff is required.
D2	chit-chat filter is exact-match on 8 template phrases; paraphrased pleasantries are kept in the intent	Low	Black-box error guessing on paraphrased pleasantries in intent turns	Open: consolidation is rule-based (8 templates). Mitigation: model still reads the remaining intent; learned filter is future work.
D-DATA-1	documented counts are 11,900/5,076/1,024 hunks per language; released data differs	Low	Data pipeline test vs documented per-language hunk counts	Open: documentation discrepancy; reports must quote the released counts, not the documented ones.
D-DATA-2	test split has no TypeScript although val does; TS generalisation is untested	Medium	Data pipeline test of language coverage per split	Open: test split has no TypeScript, so TS generalisation is unmeasured; stated as a limitation.

6. Framework Coverage

Framework section	Done	Not done
Performance	Test-split metrics per tier and latency are produced by experiment scripts; back-to-back agreement of student and INT8 (section 3.7).	No repeated-seed variance; no performance test under production load.
Testing Methods	Smoke, white-box, black-box (partitions, boundaries, error guessing), metamorphic (8 relations), adversarial (4 attacks plus robustness), back-to-back, UI e2e.	No full pairwise combinatorial generation; no A/B test in production; red team is one internal tester.
Data	Leakage checks by repository, session and pair; label-noise counts; determinism; dataset statistics consistency.	No human label validation; no TypeScript in the test split (D-DATA-2).
AI Models	Structure, heads, losses, calibration, teacher/student/INT8 comparison.	Explanation, distillation, GAN-baseline and artifact-export modules have no unit tests.
Context	Runs locally on CPU through the API and the UI; e2e flows exercised.	Only one machine and one OS; no integration with a real coding agent.
Documentation	This report, defect register, xfail reasons in the code, test-to-section mapping in test_summary.json.	No separate test plan signed off by a second reviewer.

7. Readiness Statement

Sift is ready as a **research prototype and local triage aid**: 214 of 220 test items pass, none fail, and the six known defects are documented. It is **not ready** as an automatic gate that blocks merges, for these reasons:

- The model can be fooled by echoed intent text (D3 High, D4 Medium; 68% and 52% in the latest run). A developer or an agent who copies the request into a comment can hide a misaligned hunk. Stripping comment-only added lines before scoring is a proposed, not implemented, mitigation.
- Coverage is 73.4% of the measured scope; training scripts are outside it and four modules are at 0% in the unit suite.
- All results come from one CPU (Intel Core i7-13650HX), one run and constructed labels without human validation; the test split has only Python and JavaScript.
- The e2e suite is small (37 tests in the aggregated run); plain `diff -u` multi-file input is not split correctly (D1).